

IMAx3011 – komplett løysingsforslag
Utsett eksamen 12.05.2026

Oppgave 1

Original oppgåvetekst.

La

$$A = \begin{bmatrix} 1 & 1 \\ 1 & -1 \\ 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 0 \\ 2 \\ 0 \end{bmatrix}.$$

- a) (2 poeng) Bruk definisjonen av ortogonalitet for å vise at kolonnene i A er ortogonale.
- b) (1.5 poeng) Normaliser kolonnene i A og finn en ortonormal basis for $\text{Col}(A)$.
- c) (1.5 poeng) Finn projeksjonen av b på $\text{Col}(A)$.
- d) (1 poeng) Sett $r = b - \hat{b}$, der $\hat{b}$ er projeksjonen fra deloppgave c). Vis at r er ortogonal på kolonnene i A .

Vi skriv kolonnene i A som

$$a_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad a_2 = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}.$$

- a) Kolonnene er ortogonale dersom indreproduktet er null. Her får vi

$$a_1^T a_2 = 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 + 1 \cdot (-1) = 0.$$

Dermed er kolonnene i A ortogonale.

- b) Normene er

$$\|a_1\| = \sqrt{1 + 1 + 1 + 1} = 2, \quad \|a_2\| = 2.$$

En ortonormal basis for $\text{Col}(A)$ er difor

$$q_1 = \frac{1}{2} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad q_2 = \frac{1}{2} \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix}.$$

c) Sidan q_1, q_2 er ortonormale, er projeksjonen

$$\hat{b} = (q_1^T b)q_1 + (q_2^T b)q_2.$$

Vi har

$$q_1^T b = \frac{1}{2}(2 + 0 + 2 + 0) = 2, \quad q_2^T b = \frac{1}{2}(2 - 0 + 2 - 0) = 2.$$

Dermed

$$\hat{b} = 2q_1 + 2q_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 2 \\ 0 \end{bmatrix}.$$

Altså er $\hat{b} = b$.

d) Restvektoren er

$$r = b - \hat{b} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Nullvektoren er ortogonal på alle vektorar, så spesielt er

$$r^T a_1 = 0, \quad r^T a_2 = 0.$$

Hint til sensor. Gje utteljing for korrekt bruk av indreprodukt. I c) kan kandidaten òg bruke formelen $A(A^T A)^{-1} A^T b$. Her er eit viktig poeng at b allereie ligg i kolonnerommet.

Oppgave 2

Original oppgåvetekst.

Gitt matrisen

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 3 & b \\ 1 & b & 2 \end{bmatrix}.$$

- a) (2 poeng) Bestem for hvilke verdier av b matrisen er positiv definit.
- b) (1 poeng) Bruk Python til å plote den minste egenverdien til A som funksjon av b . Velg et passende intervall for b . Bruk gjerne ditt svar fra a) som utgangspunkt i valg av intervallet.
- c) (1 poeng) Bruk plottet til å forklare når Cholesky-faktorisering eksisterer. (Det er vanskelig å kopiere matplotlib-bildet inn i Inspira, men lag gjerne en grov kopi på papir med utgangspunktet i det du ser på skjermen). Får du ikke til plottet, bruk utregningene fra deloppgave a) til å forklare når Cholesky-faktorisering eksisterer og hva slags krav setter dette for den minste egenverdien.
- d) (1 poeng) Finn Cholesky-faktoriseringen for én valgt verdi av b der faktoriseringen eksisterer.

Tips: Sylvesters kriterium sier at en symmetrisk matrise er positiv definit hvis og bare hvis alle de ledende hovedminorene er positive.

For en 3×3 -matrise betyr dette at du må sjekke:

$$\Delta_1 = \det([a_{11}]) > 0$$

$$\Delta_2 = \det \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} > 0$$

$$\Delta_3 = \det(A) > 0$$

Altså: Start øverst til venstre, og beregn determinantene til den første 1×1 -blokken, deretter den øverste venstre 2×2 -blokken, og til slutt hele matrisen. Hvis alle tre er positive, er matrisen positiv definit. Du kan bruke at $\det(A) = -4b^2 + 4b + 13$.

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 3 & b \\ 1 & b & 2 \end{bmatrix}.$$

- a) Sylvesters kriterium gir at A er positiv definit dersom alle leiande hovudminorar er positive:

$$\Delta_1 = 4 > 0,$$

$$\Delta_2 = \det \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix} = 12 - 4 = 8 > 0,$$

og

$$\Delta_3 = \det(A) = -4b^2 + 4b + 13.$$

Vi treng difor

$$-4b^2 + 4b + 13 > 0.$$

Nullpunkta er

$$b = \frac{1 \pm \sqrt{14}}{2}.$$

Sidan parabelen vender nedover, er A positiv definit for

$$\frac{1 - \sqrt{14}}{2} < b < \frac{1 + \sqrt{14}}{2}.$$

Omtrent er dette

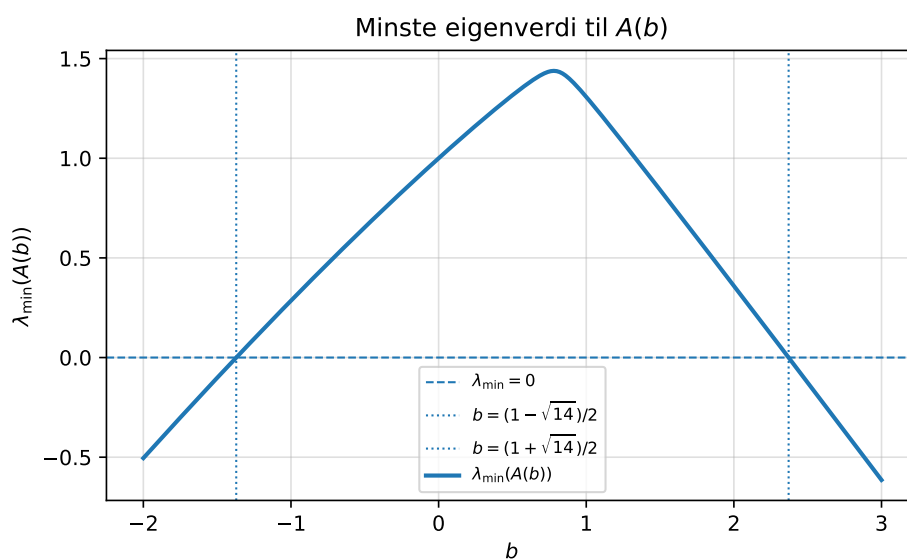
$$-1.3708 < b < 2.3708.$$

b) Eit komplett Python-program som plottar minste eigenverdi som funksjon av b er:

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4
5  def A_of_b(b):
6      return np.array([
7          [4.0, 2.0, 1.0],
8          [2.0, 3.0, b],
9          [1.0, b, 2.0]
10     ])
11
12  bs = np.linspace(-2.0, 3.0, 501)
13  lambda_min = np.array([np.linalg.eigvalsh(A_of_b(b))[0] for b in bs])
14
15  plt.axhline(0.0, linestyle="--")
16  plt.plot(bs, lambda_min)
17  plt.xlabel("b")
18  plt.ylabel("minste eigenverdi")
19  plt.title("Minste eigenverdi til A(b)")
20  plt.grid(True)
21  plt.show()

```



Figur 1: Minste eigenverdi $\lambda_{\min}(A(b))$ for $-2 \leq b \leq 3$. Dei stipla vertikale linjene markerer endepunkta $b = (1 \pm \sqrt{14})/2$.

- c) For ei reell symmetrisk matrise eksisterer Cholesky-faktoriseringa $A = LL^T$ med positiv diagonal i L dersom og berre dersom A er positiv definit. Plottet skal vise at den minste eigenverdien er positiv akkurat i intervallet

$$\frac{1 - \sqrt{14}}{2} < b < \frac{1 + \sqrt{14}}{2}.$$

Ved endepunkta er minste eigenverdi null, og utanfor intervallet er han negativ. Dermed eksisterer Cholesky-faktoriseringa akkurat i dette opne intervallet.

- d) Vel til dømes $b = 1$, som ligg i intervallet. Då er

$$A = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 3 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Cholesky-faktoriseringa $A = LL^T$ er

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & \sqrt{2} & 0 \\ \frac{1}{2} & \frac{1}{2\sqrt{2}} & \sqrt{\frac{13}{8}} \end{bmatrix}.$$

Eit komplett program som reproduserer faktoriseringa numerisk er:

```
1 import numpy as np
2
3 b = 1.0
4 A = np.array([
5     [4.0, 2.0, 1.0],
6     [2.0, 3.0, b],
7     [1.0, b, 2.0]
8 ])
9
10 L = np.linalg.cholesky(A)
11 print(L)
12 print(L @ L.T)
```

Hint til sensor. Det viktigaste i a) er korrekt ulikskap for determinanten. I b)–c) bør sensor godta både numerisk plott og forklaring frå Sylvesters kriterium. I d) kan kandidaten velje ein annan b så lenge b ligg i intervallet og faktoriseringa er rett.

Oppgave 3

Original oppgåvetekst.

Vi vil tilnærme løsningen av det lineære systemet $Ax = b$, der

$$A = \begin{bmatrix} 4 & 1 & -1 \\ 2 & 7 & 1 \\ 1 & -3 & 12 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 19 \\ 31 \end{bmatrix}.$$

Betrakt følgende Python-funksjon:

```

1  import numpy as np
2
3  def solver(A, b, x0, n_iter):
4      n = x0.size
5      x = x0.copy()
6      for k in range(n_iter):
7          for i in range(n):
8              s1 = A[i, :i] @ x[:i]
9              s2 = A[i, i+1:] @ x[i+1:]
10             x[i] = (b[i] - s1 - s2) / A[i, i]
11     return x

```

- (1 poeng) Er dette Jacobi-metoden eller Gauss–Seidel-metoden? Begrunn kort ut fra hvordan vektoren x oppdateres.
- (1 poeng) Forklar hvorfor matrisen A har en egenskap som gjør at metoden konvergerer.
- (2 poeng) Bruk startvektor $x^{(0)} = (0, 0, 0)^T$ og utfør to iterasjoner for hånd. Oppgi $x^{(1)}$ og $x^{(2)}$, avrundet til 4 desimaler.
- (1 poeng) Den eksakte løsningen til systemet er $x = (1, 2, 3)^T$. Sammenlign kort denne løsningen med iteratene dine.

Python-funksjonen er

```

1  import numpy as np
2
3  def solver(A, b, x0, n_iter):
4      n = x0.size
5      x = x0.copy()
6      for k in range(n_iter):
7          for i in range(n):
8              s1 = A[i, :i] @ x[:i]
9              s2 = A[i, i+1:] @ x[i+1:]
10             x[i] = (b[i] - s1 - s2) / A[i, i]
11     return x

```

- Dette er Gauss–Seidel-metoden. Grunnen er at komponentane i x blir oppdaterte éin og éin, og nye verdier blir brukte straks dei er rekna ut. I linja

```
s1 = A[i, :i] @ x[:i]
```

bruker koden dei oppdaterte komponentane $x_1, \dots, x_{i-1}$ frå same iterasjon.

b) Matrisen er strengt diagonaldominant radvis:

$$|4| > |1| + |-1| = 2,$$

$$|7| > |2| + |1| = 3,$$

$$|12| > |1| + |-3| = 4.$$

For strengt diagonaldominante matriser konvergerer Gauss–Seidel-metoden.

c) Start med $x^{(0)} = (0, 0, 0)^T$. Første iterasjon:

$$x_1^{(1)} = \frac{3 - 0 + 0}{4} = 0.75,$$

$$x_2^{(1)} = \frac{19 - 2(0.75) - 0}{7} = 2.5,$$

$$x_3^{(1)} = \frac{31 - 0.75 + 3(2.5)}{12} = 3.145833\dots$$

Altså

$$x^{(1)} \approx (0.7500, 2.5000, 3.1458)^T.$$

Andre iterasjon:

$$x_1^{(2)} = \frac{3 - x_2^{(1)} + x_3^{(1)}}{4} = \frac{3 - 2.5 + 3.145833\dots}{4} = 0.911458\dots,$$

$$x_2^{(2)} = \frac{19 - 2x_1^{(2)} - x_3^{(1)}}{7} = 2.004464\dots,$$

$$x_3^{(2)} = \frac{31 - x_1^{(2)} + 3x_2^{(2)}}{12} = 3.008495\dots$$

Dermed

$$x^{(2)} \approx (0.9115, 2.0045, 3.0085)^T.$$

d) Den eksakte løysinga er $(1, 2, 3)^T$. Allereie etter to iterasjonar er andre og tredje komponent svært nær, medan første komponent enno ligg om lag 0.0885 under den eksakte verdien. Iterata nærmar seg den eksakte løysinga, som venta.

Eit komplett program som køyrer to iterasjonar er:

```
1 import numpy as np
2
3 A = np.array([
4     [4.0, 1.0, -1.0],
5     [2.0, 7.0, 1.0],
6     [1.0, -3.0, 12.0]
7 ])
8 b = np.array([3.0, 19.0, 31.0])
9 x0 = np.array([0.0, 0.0, 0.0])
10
11 x = solver(A, b, x0, 2)
12 print(x)
13 print("eksakt:", np.array([1.0, 2.0, 3.0]))
```

Hint til sensor. For full uttelling i a) må kandidaten skilje klart mellom Jacobi og Gauss–Seidel. I c) er små avrundingsavvik uproblematisk dersom reknegangen er rett.

Oppgave 4

Original oppgåvetekst.

Vi betrakter funksjonen

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - x_1^2)^2.$$

- a) (2 poeng) Finn gradienten $\nabla f(x_1, x_2)$.
- b) (2 poeng) Fullfør kodesnutten nedenfor slik at `scipy.optimize.minimize` bruker BFGS med gradient.

```

1  import numpy as np
2  from scipy.optimize import minimize
3
4  def f(x):
5      return (x[0]-1)**2 + 5*(x[1]-x[0]**2)**2
6
7  def gradf(x):
8      return np.array([
9          ???,
10         ???
11     ])
12
13  x0 = np.array([0.0, 0.0])
14  result = minimize(f, x0, jac=???, method=???)
15  print(result.x, result.fun)
```

- c) (1 poeng) Hvilket minimumspunkt og hvilken minimumsverdi forventer du at algoritmen finner? Begrunn kort uten å kjøre koden.
- d) (1 poeng) Forklar kort hvorfor BFGS vanligvis er mer effektiv enn Nelder–Mead for denne funksjonen.

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - x_1^2)^2.$$

- a) Vi deriverer komponentvis. For x_1 får vi

$$\frac{\partial f}{\partial x_1} = 2(x_1 - 1) + 10(x_2 - x_1^2)(-2x_1) = 2(x_1 - 1) - 20x_1(x_2 - x_1^2).$$

For x_2 får vi

$$\frac{\partial f}{\partial x_2} = 10(x_2 - x_1^2).$$

Dermed

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2(x_1 - 1) - 20x_1(x_2 - x_1^2) \\ 10(x_2 - x_1^2) \end{bmatrix}.$$

- b) Ferdig kode:

```

1  import numpy as np
2  from scipy.optimize import minimize
3
4
```



```
5 def f(x):
6     return (x[0] - 1)**2 + 5*(x[1] - x[0]**2)**2
7
8
9 def gradf(x):
10    return np.array([
11        2*(x[0] - 1) - 20*x[0]*(x[1] - x[0]**2),
12        10*(x[1] - x[0]**2)
13    ])
14
15 x0 = np.array([0.0, 0.0])
16 result = minimize(f, x0, jac=gradf, method="BFGS")
17 print(result.x, result.fun)
```

- c) Sidan begge ledda i f er kvadratledd, er $f(x_1, x_2) \geq 0$. Minimumsverdien 0 blir oppnådd når

$$x_1 - 1 = 0, \quad x_2 - x_1^2 = 0.$$

Dette gir

$x^* = (1, 1)^T, \quad f(x^*) = 0.$

- d) BFGS bruker gradientinformasjon og byggjer opp ei tilnærming til invers Hessian. For ei glatt funksjon som denne gir det vanlegvis raskare og meir retningsinformerte steg enn Nelder–Mead, som berre bruker funksjonsverdiar og difor ofte treng fleire evalueringar.

Hint til sensor. I b) er det viktig at ‘jac=gradf’ og ‘method=BFGS’ er med. Godta ekvivalente uttrykk for gradienten.

Oppgave 5

Original oppgåvetekst.

La

$$f(x_1, x_2) = x_1^2 + 2x_2^2 - 2x_1 + 4x_2, \quad x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad p = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

- a) (1 poeng) Sett opp funksjonen $\phi(\alpha) = f(x_0 + \alpha p)$.
- b) (2 poeng) Finn den perfekte steglengden α^* som minimerer $\phi(\alpha)$.
- c) (1 poeng) Finn det nye punktet $x_1 = x_0 + \alpha^* p$ og beregn $f(x_1)$.

$$f(x_1, x_2) = x_1^2 + 2x_2^2 - 2x_1 + 4x_2, \quad x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad p = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

- a) Vi har

$$x_0 + \alpha p = \begin{bmatrix} \alpha \\ -\alpha \end{bmatrix}.$$

Dermed

$$\phi(\alpha) = f(\alpha, -\alpha) = \alpha^2 + 2\alpha^2 - 2\alpha - 4\alpha = 3\alpha^2 - 6\alpha.$$

- b) Deriver:

$$\phi'(\alpha) = 6\alpha - 6.$$

Set $\phi'(\alpha) = 0$:

$$6\alpha - 6 = 0 \quad \Rightarrow \quad \boxed{\alpha^* = 1}.$$

Sidan $\phi''(\alpha) = 6 > 0$, er dette eit minimum.

- c) Det nye punktet er

$$x_1 = x_0 + \alpha^* p = \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Funksjonsverdien er

$$f(1, -1) = 1^2 + 2(-1)^2 - 2 \cdot 1 + 4(-1) = 1 + 2 - 2 - 4 = -3.$$

Altså

$$\boxed{x_1 = (1, -1)^T, f(x_1) = -3.}$$

Hint til sensor. Kandidaten bør få uttelling for korrekt éindimensjonal funksjon og for å kontrollere at kritisk punkt faktisk er minimum.

Oppgave 6

Original oppgåvetekst.

Vi ønsker å løse likningen

$$x = \cos(x)$$

ved fikspunktiterasjon.

- a) (2 poeng) Bruk iterasjonen

$$x_{n+1} = \cos(x_n), \quad x_0 = 1,$$

og finn x_1 , x_2 og x_3 .

- b) (1 poeng) La $g(x) = \cos(x)$ og anta at $\alpha \approx 0.739$ er fikspunktet. Bruk $g'(x)$ til å forklare kort hvorfor iterasjonen er lokalt konvergent.
- c) (2 poeng) Betrakt den modifiserte iterasjonen

$$x_{n+1} = \frac{x_n + \cos(x_n)}{2}.$$

Finn deriverte til denne iterasjonsfunksjonen og sammenlign den asymptotiske konvergensthastigheten (dvs hvor fort den konvergerer for store n) med den opprinnelige metoden nær α .

Hint: Hvis iterasjonen skrives som $x_{n+1} = g(x_n)$ og

$$|g'(\alpha)| < 1,$$

så er metoden lokalt konvergent mot fikspunktet α . Lar vi

$$e_n = |x_n - \alpha|$$

være feilen i iterasjon n , så gjelder omtrent

$$e_{n+1} \approx |g'(\alpha)| e_n,$$

så mindre verdi av $|g'(\alpha)|$ gir raskere konvergens.

Vi skal løyse $x = \cos x$ med fikspunktiterasjon.

- a) Med $x_0 = 1$ får vi

$$x_1 = \cos(1) = 0.540302\dots,$$

$$x_2 = \cos(x_1) = \cos(0.540302\dots) = 0.857553\dots,$$

$$x_3 = \cos(x_2) = \cos(0.857553\dots) = 0.654290\dots$$

Altså

$$x_1 \approx 0.5403, \quad x_2 \approx 0.8576, \quad x_3 \approx 0.6543.$$

- b) Her er $g(x) = \cos x$, så

$$g'(x) = -\sin x.$$

Ved fikspunktet $\alpha \approx 0.739$ er

$$|g'(\alpha)| = |\sin(\alpha)| \approx 0.6735 < 1.$$

Dermed er iterasjonen lokalt konvergent.

c) For den modifiserte iterasjonen er

$$h(x) = \frac{x + \cos x}{2}.$$

Derivert blir

$$h'(x) = \frac{1 - \sin x}{2}.$$

Ved fikspunktet $\alpha \approx 0.739$ får vi

$$|h'(\alpha)| = \frac{1 - \sin(\alpha)}{2} \approx 0.1632.$$

Dette er mykje mindre enn $|g'(\alpha)| \approx 0.6735$, så den modifiserte iterasjonen har mykje raskare asymptotisk lineær konvergens nær fikspunktet.

Hint til sensor. I c) er samanlikninga av storleikane $|g'(\alpha)|$ og $|h'(\alpha)|$ hovudpoenget. Fortegn er mindre viktig enn absoluttverdien for konvergensfarten.

Oppgave 7

Original oppgåvetekst.

Et verksted lager to produkttyper, A og B . Hver enhet krever tid til kutting, montering og pakking. Tabellen viser ressursbruk og fortjeneste per enhet:

Ressurs / fortjeneste	Produkt A	Produkt B	Tilgjengelig
Kutting (min)	3	1	18
Montering (min)	1	2	14
Pakking (min)	1	1	8
Fortjeneste (kr)	70	50	–

La x være antall enheter av produkt A og y være antall enheter av produkt B . Det betyr at både x og y er heltallige og ikke-negative.

- (2 poeng) Formuler dette som et lineært optimeringsproblem mtp maksimal fortjeneste.
- (2 poeng) Finn den optimale løsningen grafisk og oppgi maksimal fortjeneste.
- (1 poeng) Hvilke bibetingelser er bindende (dvs gjelder med likhet) i optimum?
- (1 poeng) Formuler dualproblemet og tolk dualvariablene.

La x være talet på einingar av produkt A og y talet på einingar av produkt B .

- Det lineære optimeringsproblemet er

$$\max Z = 70x + 50y$$

med bibetingelsar

$$3x + y \leq 18,$$

$$x + 2y \leq 14,$$

$$x + y \leq 8,$$

$$x \geq 0, \quad y \geq 0,$$

og i den heiltalige modellen

$$x, y \in \mathbb{Z}.$$

- Hjørnepunkta i det kontinuerlege polytopet er

$$(0, 0), \quad (6, 0), \quad (5, 3), \quad (2, 6), \quad (0, 7).$$

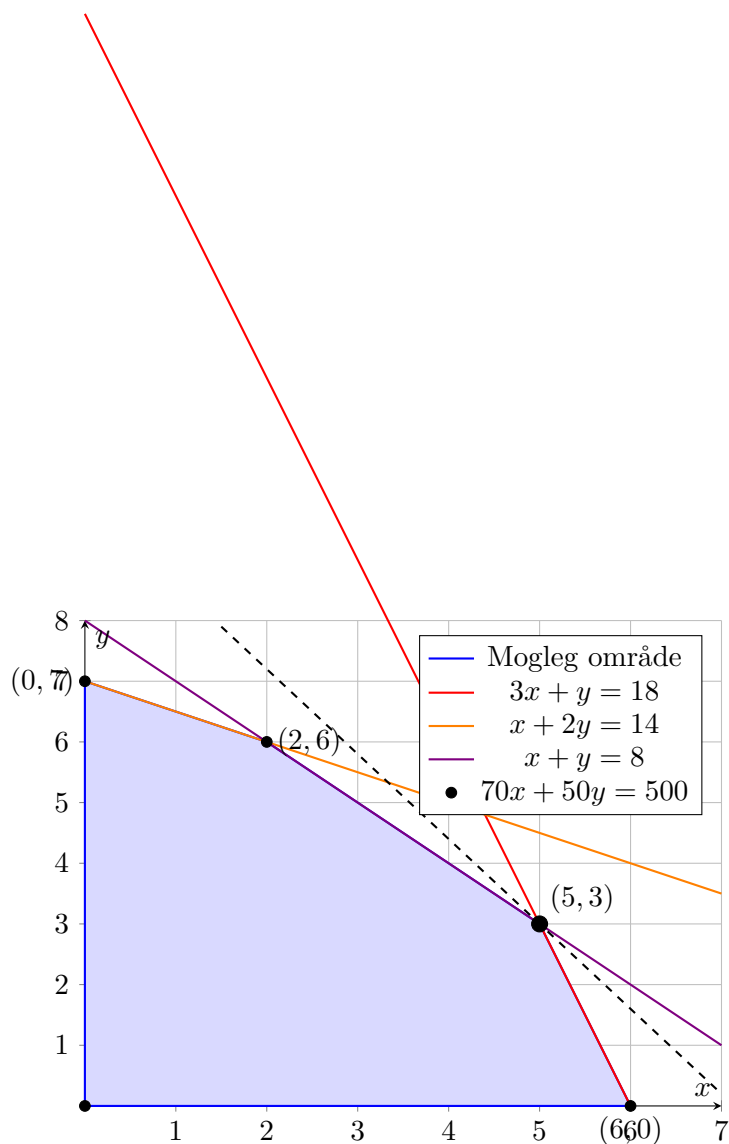
Fortenesta i desse punkta er

$$0, \quad 420, \quad 500, \quad 440, \quad 350.$$

Dermed er optimum

$$(x, y) = (5, 3), \quad Z_{\text{maks}} = 500.$$

Sidan $(5, 3)$ er heiltalig, er dette òg optimal løysing for heiltalsproblemet.



c) I optimum er

$$3x + y = 3 \cdot 5 + 3 = 18,$$

og

$$x + y = 5 + 3 = 8.$$

Desse to bibetingelsane er bindande. Monteringskravet er ikkje bindande, sidan

$$x + 2y = 5 + 2 \cdot 3 = 11 < 14.$$

d) Dualproblemet til den kontinuerlege LP-relakseringa er

$$\min W = 18u_1 + 14u_2 + 8u_3$$

med

$$3u_1 + u_2 + u_3 \geq 70,$$

$$u_1 + 2u_2 + u_3 \geq 50,$$

$$u_1, u_2, u_3 \geq 0.$$

Dualvariablane u_1, u_2, u_3 kan tolkast som skuggeprisar for eitt ekstra minutt kutting, montering og pakking. Sidan monteringskravet ikkje er bindande i optimum, vil ein vente $u_2 = 0$ i ei komplementær optimal dual-løysing.

Hint til sensor. Dersom kandidaten tek heiltalskravet bokstaveleg, er det likevel nok å undersøkje heiltalige punkt eller argumentere via det grafiske optimumet, sidan optimumet er heiltalig. I d) er det naturleg å formulere dualen for LP-relakseringa.

Oppgave 8

Original oppgåvetekst.

Vi bruker et desimalt flyttallssystem med **4 signifikante sifre** og avrunding til nærmeste representerbare tall.

- (1.5 poeng) Lagre tallene $x = 12.345$ og $y = 12.344$ i dette systemet. Beregn deretter den maskinelle differansen $\text{fl}(x - y)$ og sammenlign med den eksakte differansen $x - y$.
- (1 poeng) Forklar kort hvorfor den relative feilen i differansen kan bli stor selv om både x og y hver for seg er lagret med liten relativ feil.
- (1.5 poeng) For små $t > 0$ ønsker vi å beregne

$$\sqrt{1+t} - 1.$$

Vis at uttrykket kan skrives om til

$$\frac{t}{\sqrt{1+t} + 1},$$

og forklar hvorfor denne formen vanligvis er numerisk mer stabil.

Vi bruker eit desimalt flyttallssystem med fire signifikante siffer og avrunding til næraste representerbare tal.

- Lagra verdier blir

$$\text{fl}(12.345) = 12.35, \quad \text{fl}(12.344) = 12.34.$$

Den maskinelle differansen blir dermed

$$\text{fl}(x) - \text{fl}(y) = 12.35 - 12.34 = 0.01.$$

Med fire signifikante siffer er 0.01 representerbart, så

$$\text{fl}(\text{fl}(x) - \text{fl}(y)) = 0.01000.$$

Den eksakte differansen er

$$x - y = 12.345 - 12.344 = 0.001.$$

Den maskinelle differansen er altså ti ganger for stor.

- Grunnen er kansellering. Tala x og y er nesten like, så når dei blir trekte frå kvarandre, forsvinn dei leiande siffera. Små absolutte avrundingsfeil i x og y kan då bli store samanlikna med den vesle differansen.
- Vi rasjonaliserer uttrykket:

$$\sqrt{1+t} - 1 = \frac{(\sqrt{1+t} - 1)(\sqrt{1+t} + 1)}{\sqrt{1+t} + 1} = \frac{1+t-1}{\sqrt{1+t} + 1} = \frac{t}{\sqrt{1+t} + 1}.$$

For små $t > 0$ er $\sqrt{1+t}$ nær 1, så uttrykket $\sqrt{1+t} - 1$ trekkjer frå to nesten like tal. Det gir risiko for kansellering. Forma

$$\frac{t}{\sqrt{1+t} + 1}$$

unngår denne subtraksjonen og er difor vanlegvis meir numerisk stabil.

Hint til sensor. Her bør sensor sjå etter at kandidaten faktisk lagrar tala først og deretter reknar differansen. Det er sentralt at feilen kjem frå subtraksjon av avrunda nesten-like tal.

Oppgave 9

Original oppgåvetekst.

La

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

- a) (1 poeng) Finn egenverdiene til A og en egenvektor til den dominante egenverdien.
- b) (1.5 poeng) Bruk potensmetoden med startvektor $x_0 = (1, 0)^T$. Utfør to iterasjoner der du normaliserer i hver iterasjon med 2-normen. Oppgi de normaliserte vektorene u_1 og u_2 .
- c) (0.5 poeng) Bruk Rayleigh-kvotienten til å gi et estimat av dominant egenverdi basert på u_2 .
- d) (1 poeng) Forklar hvorfor potensmetoden ikke vil konvergere mot en dominant egenvektor dersom man velger startvektor $x_0 = (1, -1)^T$.

$$A = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}.$$

- a) Karakteristisk polynom er

$$\det(A - \lambda I) = \det \begin{bmatrix} 3 - \lambda & 1 \\ 1 & 3 - \lambda \end{bmatrix} = (3 - \lambda)^2 - 1.$$

Dermed

$$(3 - \lambda)^2 = 1,$$

så

$$\lambda_1 = 4, \quad \lambda_2 = 2.$$

Den dominante egenverdien er 4, med til dømes egenvektor

$$v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

- b) Start med $x_0 = (1, 0)^T$. Første steg:

$$Ax_0 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}, \quad \|Ax_0\|_2 = \sqrt{10}.$$

Difor

$$u_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 3 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.9487 \\ 0.3162 \end{bmatrix}.$$

Andre steg:

$$Au_1 = \frac{1}{\sqrt{10}} \begin{bmatrix} 10 \\ 6 \end{bmatrix}.$$

Når vi normaliserer, kan faktoren $1/\sqrt{10}$ strykast:

$$\|(10, 6)^T\|_2 = \sqrt{136} = 2\sqrt{34}.$$

Dermed

$$u_2 = \frac{1}{\sqrt{136}} \begin{bmatrix} 10 \\ 6 \end{bmatrix} = \frac{1}{\sqrt{34}} \begin{bmatrix} 5 \\ 3 \end{bmatrix} \approx \begin{bmatrix} 0.8575 \\ 0.5145 \end{bmatrix}.$$

c) Rayleigh-kvotienten basert på u_2 er

$$\rho(u_2) = \frac{u_2^T A u_2}{u_2^T u_2}.$$

Sidan u_2 er normalisert, er nemnaren 1. Med $u_2 = (5, 3)^T / \sqrt{34}$ får vi

$$\rho(u_2) = \frac{1}{34} \begin{bmatrix} 5 & 3 \end{bmatrix} \begin{bmatrix} 18 \\ 14 \end{bmatrix} = \frac{90 + 42}{34} = \frac{132}{34} = \frac{66}{17} \approx 3.8824.$$

d) Startvektoren $(1, -1)^T$ er ein eigenvektor til eigenverdien 2, fordi

$$A \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ -2 \end{bmatrix} = 2 \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

Han har ingen komponent i retning av den dominante eigenvektoren $(1, 1)^T$. Difor vil potensmetoden halde seg i eigenrommet til eigenverdien 2 og ikkje konvergere mot den dominante eigenvektoren.

Hint til sensor. I b) må normalisering med 2-norm kome tydeleg fram. I d) er hovudpoenget at startvektoren er ortogonal på dominant eigenvektor og ligg nøyaktig i det andre eigenrommet.